

Denotational Semantics

- If $\langle S, \sigma \rangle \rightarrow^* \langle E, \tau \rangle$ (or in other words, if program S in state σ terminate in τ) then we say τ is the **denotational semantics of S in σ** , and we write $M(S, \sigma) = \{\tau\}$.
 - We can consider the denotational semantics of a program as the final evaluation of a program. It is like we skip all details in the operational semantics.
 - We can sometimes simplify $\{\tau\}$ to τ , but technically it is not correct, because a configuration might end up with a collection of different states (we will see that next week). To sum up, we can simplify $\{\tau\}$ to τ , unless $\tau = \emptyset$ or τ is more than one state.

(Denotational Semantics Rules)

Denotational semantics rules are basically just operational semantics rules written in a different way.

- $M(\mathbf{skip}, \sigma) = \{\sigma\}$
- $M(v := e, \sigma) = \{\sigma[v \mapsto \sigma(e)]\}$
- $M(b[e_1] := e, \sigma) = \{\sigma[b[\sigma(e_1)] \mapsto \sigma(e)]\}$
- $M(\mathbf{if } B \mathbf{ then } S_1 \mathbf{ else } S_2 \mathbf{ fi}, \sigma) = \begin{cases} M(S_1, \sigma), & \text{if } \sigma(B) = T \\ M(S_2, \sigma), & \text{if } \sigma(B) = F \end{cases}$
- Let $W \equiv \mathbf{while } B \mathbf{ do } S \mathbf{ od}$, then $M(W, \sigma) = \begin{cases} \{\sigma\}, & \text{if } \sigma(B) = F \\ M(S; W, \sigma) = M(W, M(S, \sigma)), & \text{if } \sigma(B) = T \end{cases}$
- $M(S_1; S_2, \sigma) = M(S_2, M(S_1, \sigma))$
 - Note that, function M can be defined on a collection of states as well, we will discuss that in future classes.

1. Let $IF \equiv \mathbf{if } n < 0 \mathbf{ then } y := 0 \mathbf{ else } x := 0; y := 1; W \mathbf{ fi}$, where $W \equiv \mathbf{while } x < n \mathbf{ do } x := x + 1; y := y + 1 \mathbf{ od}$ calculate $M(IF, \sigma)$ where $\sigma = \{n = 1\}$.

$$\begin{aligned}
 M(IF, \sigma) &= M(x := 0; y := 1; W, \{n = 1\}) \\
 &= M(y := 1; W, \{n = 1, x = 0\}) \\
 &= M(W, \{n = 1, x = 0, y = 1\}) \\
 &= M(W, M(x := x + 1; y := y + 1, \{n = 1, x = 0, y = 1\})) \\
 &= M(W, M(y := y + 1, \{n = 1, x = 1, y = 1\})) \\
 &= M(W, \{n = 1, x = 1, y = 2\}) \quad // \text{Note that, the inner } M \text{ outputs one state here} \\
 &= \{\{n = 1, x = 1, y = 2\}\}
 \end{aligned}$$

Divergence

We learned that if S in σ converges to τ , we have $\langle S, \sigma \rangle \rightarrow^* \langle E, \tau \rangle$ and $M(S, \sigma) = \{\tau\}$; but what if S diverges?

- We define pseudo-state " $\perp$ " (reads "bottom") to represent a program cannot terminate successfully.
 - $\perp$ is not a real state in memory.
 - There can be multiple reasons a program cannot terminate successfully, such as the program diverges, or the program meets some runtime error and halts.
- Denotationally, we write $M(S, \sigma) = \{\perp_d\}$ to represent S diverges in σ . Operationally, $\langle S, \sigma \rangle \rightarrow^* \langle E, \perp_d \rangle$ means that S starting in σ diverges.

- Here, we present two situations that we can show S diverges in σ . However, if program S and state σ doesn't satisfy the following situation, it is still possible that S diverges in σ ; in general, "Whether arbitrary S converges in arbitrary σ ?" is an undecidable problem.
 - While evaluating an iterative statement $W \equiv \mathbf{while\ } B \mathbf{ do\ } S \mathbf{ od}$, and get a sequence $\langle W, \sigma_0 \rangle \rightarrow^* \langle W, \sigma_1 \rangle \rightarrow^* \langle W, \sigma_2 \rangle \rightarrow \dots$, if we can prove that $\neg \exists i. \sigma_i(B) = F$, then W in σ_0 diverges.
 - A special case that we can prove an iterative statement diverges is when there is a cycle in the operational semantic evaluation: in the sequence of configurations $\langle S_0, \sigma_0 \rangle \rightarrow \langle S_1, \sigma_1 \rangle \rightarrow \langle S_2, \sigma_2 \rangle \rightarrow \dots$, if $\exists i. \exists j. i \neq j \wedge S_i = S_j \wedge \sigma_i = \sigma_j$, then S_0 starting in σ_0 diverges.
- 2. Evaluate $W \equiv \mathbf{while\ } T \mathbf{ do\ skip\ od}$ in σ .
 - Since the loop condition is always evaluated to T , thus $M(W, \sigma) = \{\perp_d\}$. Or operationally, we can write $\langle W, \sigma \rangle \rightarrow^* \langle E, \perp_d \rangle$.
 - Another way to show W diverges in σ is showing that: $\langle W, \sigma \rangle \rightarrow \langle \mathbf{skip}; W, \sigma \rangle \rightarrow \langle W, \sigma \rangle$; we got a cycle in the evaluation.

Runtime Errors

- Like divergence, we use a pseudo-state (also a pseudo-value) " $\perp_e$ " to represent the state (or value) when a runtime error happens.
- Runtime errors can happen in both expressions and statements. Let's look at runtime errors in expressions first. We write $\sigma(e) = \perp_e$ to represent that the evaluation of expression e in state σ causes a runtime error. Here " $\perp_e$ " is a pseudo-value, which means it is not a real value, it is used to represent values such as $x/0$ or $\text{sqrt}(-1)$.
 - If e might cause runtime error, then instead of $\sigma(e) \in V$ for some set value set V , we now have $\sigma(e) \in V \cup \{\perp_e\}$ as the range of $\sigma(e)$. For example, since x/y might cause a runtime error, then its value in some state σ might result in $\perp_e$, thus we write $\sigma(x/y) \in \mathbb{Z} \cup \{\perp_e\}$.
- Types of runtime errors:
 - **Primary Failures:** The primitive values and operations being supported determine some set of basic runtime errors.
 - Array index out of bounds: $\sigma(b[e]) = \perp_e$ if $\sigma(e) < 0$ or $\sigma(e) \geq \text{size}(b)$. Similar situation for multi-dimensional arrays.
 - Division by zero: $\sigma(e_1/e_2) = \perp_e$ and $\sigma(e_1 \% e_2) = \perp_e$ if $\sigma(e_2) = 0$.
 - Square root of negative number: $\sigma(\text{sqrt}(e)) = \perp_e$ if $\sigma(e) < 0$.
 - **Hereditary Failure:** If evaluating a subexpression fails, then the overall expression fails.
 - Let op be a unary operator; then $\sigma(op\ e) = \perp_e$ if $\sigma(e) = \perp_e$. For example: $e \equiv x/0$, then $\sigma(e) = \perp_e$, and $\sigma(\text{sqrt}(e)) = \perp_e$ as well.
 - Let op be a binary operator; then $\sigma(e_1\ op\ e_2) = \perp_e$ if $\sigma(e_1) = \perp_e$ or $\sigma(e_2) = \perp_e$.
 - For a conditional expression, $\sigma(\mathbf{if\ } B \mathbf{ then\ } e_1 \mathbf{ else\ } e_2 \mathbf{ fi}) = \perp_e$ if one of the following three situations occurs:
 - 1) $\sigma(B) = \perp_e$
 - 2) $\sigma(B) = T$ and $\sigma(e_1) = \perp_e$
 - 3) $\sigma(B) = F$ and $\sigma(e_2) = \perp_e$

3. What are the states that will cause runtime errors for each of the following expressions?
 - a. $b[x/y]$ Some state σ where $\sigma(y) = 0$, or $\sigma(x/y) < 0$, or $\sigma(x/y) \geq \text{size}(b)$.
 - b. $\text{sqrt}(x) + \text{sqrt}(x/y)$ Some state σ where $\sigma(x) < 0$, or $\sigma(y) = 0$, or $\sigma(x/y) < 0$.
 - c. **if** $y = 0$ **then** 0 **else** x/y **fi** No such states.
- A runtime error in an expression can cause the statement it appears in to halt unsuccessfully. We write $\langle S, \sigma \rangle \rightarrow \langle E, \perp_e \rangle$ for the operational semantics of such a statement.
 - Here $\perp_e$ is a pseudo-state not a pseudo-value. Decide whether $\perp_e$ is a pseudo-state or pseudo-value from the context.
- With runtime errors, let's expand our operations semantics rules:
 - $\sigma(e) = \perp_e$, then $\langle v := e, \sigma \rangle \rightarrow \langle E, \perp_e \rangle$.
 - $\sigma(b[e_1]) = \perp_e$ or $\sigma(e_2) = \perp_e$, then $\langle b[e_1] := e_2, \sigma \rangle \rightarrow \langle E, \perp_e \rangle$.
 - If $\sigma(B) = \perp_e$, then $\langle \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma \rangle \rightarrow \langle E, \perp_e \rangle$.
 - On the other hand, if $\sigma(B) \neq \perp_e$, we will continue to evaluate configuration $\langle S_1, \sigma \rangle$ or $\langle S_2, \sigma \rangle$.
 - If $\langle S_1, \sigma \rangle \rightarrow \langle E, \perp_e \rangle$ then $\langle S_1; S_2, \sigma \rangle \rightarrow \langle E, \perp_e \rangle$.
 - If $\sigma(B) = \perp_e$, then $\langle \text{while } B \text{ do } S \text{ od}, \sigma \rangle \rightarrow \langle E, \perp_e \rangle$.
 - On the other hand, if $\sigma(B) \neq \perp_e$, we will continue to evaluate configuration $\langle E, \sigma \rangle$ or $\langle S; \text{while } B \text{ do } S \text{ od}, \sigma \rangle$.
4. Evaluate $S \equiv x := 0; y := 1; W$ where $W \equiv \text{while } x/y \geq 0 \text{ do } y := \text{sqrt}(y) - 1 \text{ od}$ in state σ .

$\langle S, \sigma \rangle$	$\rightarrow^* \langle W, \sigma[x \mapsto 0][y \mapsto 1] \rangle$	//before the first iteration of W
	$\rightarrow^* \langle W, \sigma[x \mapsto 0][y \mapsto 0] \rangle$	//since $\sigma[x \mapsto 0][y \mapsto 1](x/y \geq 0) = T$
	$\rightarrow \langle E, \perp_e \rangle$	//since $\sigma[x \mapsto 0][y \mapsto 0](x/y \geq 0) = \perp_e$

Properties and Consequences of $\perp$

- $\perp$ refers generically to $\perp_d$ and/or $\perp_e$. We use $\langle S, \sigma \rangle \rightarrow^* \langle E, \perp \rangle$ when it's not important which of $\perp_d$ or $\perp_e$ can occur. Similarly, $\perp \in M(S, \sigma)$ means $\langle S, \sigma \rangle$ leads to $\perp_d$ or $\perp_e$.
- Since we use $\perp$ somewhere an actual memory state appears in evaluating a program, we want to look at other situations where a state appears and think about whether $\perp$ can be used there.
 - $\perp$ is not a well-formed state.
 - **When we say, "for all states...", "for some state...", or "for all semantic values...", "for some semantic values...", we don't include $\perp$.**
 - We cannot add a binding to $\perp$: $\perp \cup \{v = \beta\} = \perp$.
 - Consider $\perp$ as a pseudo-value, then binding it with a variable will result in a pseudo-state: $\sigma(v) \neq \perp$ and $\sigma[v \mapsto \perp] = \perp$. In other words, we cannot bind a variable with pseudo-value $\perp$.
 - Evaluating a variable or an expression in $\perp$ results in a pseudo-value $\perp$: If $\sigma = \perp$ then $\sigma(v) = \sigma(e) = \perp$. In other words, we cannot take the value of a variable or expression in $\perp$.
 - Operationally, execution halts as soon as we generate $\perp$ as a pseudo-state: $\langle S, \perp \rangle \rightarrow^0 \langle E, \perp \rangle$.
 - Denotationally, we can't run a program in pseudo-state $\perp$: $M(S, \perp) = \{\perp\}$.

Predicate Logic with the consideration of $\perp$

- A predicate and an expression with Boolean value can be similar, we can evaluate both of them into Boolean values in proper states. But,
 - A Boolean expression is a piece of code that is used to represent a Boolean value. Quantifiers cannot appear in a Boolean expression.
 - A predicate is used to describe the property of values. In the science of programming, we use a predicate to describe the precondition and postcondition of statements; they don't appear in code.
 - Here is an example we used in Lecture 1, a piece of Python code:

```
# z >= c
if x > 0:
    z += x
# z >= c
```

Here, $z \geq c$ is a predicate and $x > 0$ is an expression.

- The evaluation of a predicate can end up in errors as well. Since predicates are not part of a program, we cannot call these errors “runtime errors”, but we understand these errors in the same way as they are runtime errors. We simply use $\perp$ to represent the evaluation of a predicate terminates unsuccessfully.
- $\perp$ cannot satisfy any predicate: $\perp \not\models p$ for all p , even if p is the constant T .
- Runtime errors can appear while evaluating a predicate p . In general, we now have **three possible situations for a proper state trying to satisfy a predicate**: $\sigma \models p$ (in other words $\sigma(p) = T$), $\sigma \models \neg p$ (in other words $\sigma(p) = F$), or $\sigma(p) = \perp$.
 - Previously we have $\sigma \not\models p$ means $\sigma \models \neg p$, but this is no longer true when $\perp$ is taken into consideration. $\sigma \not\models p$ means “It is not true that $\sigma \models p$ ” and it means $\sigma \models \neg p$ or $\sigma(p) = \perp$.